02-14 Agent决策引擎设计

1、自定义决策引擎的优势

标准的 ReAct Agent 属于通用框架，它根据提示推理工具使用路径，具有一定“试探性”。但在你的场景中：

- 工具功能之间有重叠或组合关系（如生成+执行）
- 用户可能用多种说法描述相同意图
- 精确性要求高，不能随便试错调用

特性	标准 ReAct Agent	自定义决策引擎
工具调用灵活性	高	高
工具调用精准性	中等（依赖提示）	高（你控制路径）
多轮多阶段任务支持	弱（需复杂提示）	强（可拆分任务）
易调试、日志追踪	中	高
可组合策略和缓存	难	易实现

2、Agent的决策主流程

```
    def main(self, user_input):
        """agent的入口函数"""
        self.user_input = user_input
        # 在记忆中添加用户的初始输入
        self.save_to_memory({
            "role": "user",
            "content": self.user_input,
            "type": "user_input"
        })
        # 进行意图识别
        self.intent_recognition()
        # 判断意图识别的可信度
        if self.intent_result.get('confidence') < 0.5:
            # 直接使用大模型来回复问题
            response = deepseek_v3_llm.invoke(self.user_input)
            return response.content
        elif self.intent_result.get('confidence') < 0.7:
            return "抱歉，我无法精准识别到您的意图，请补全缺少的参数或者重新描述意图"
        # 识别出来了用户的意图,将识别的结果保存到记忆中
        self.save_to_memory({
            "role": "assistant",
            "content": self.intent_result,
            "type": "intent_result"
        })
        # 获取工具列表
        execution_plan = self.intent_result.get('execution_plan')
        # 输出意图识别的步骤信息
```

```
        print(f"初步分析完成，完成当前需求，需要执行{len(execution_plan)}个步骤")
        for step in execution_plan:
            print(f"步骤{step.get('step')},具体操作描述:{step.get('action')},执行的工具：{step.get('tool_name')}")
        # 将要执行的任务压入队列
        self.task_queue = execution_plan[::-1]
        # 执行规划的任务
        self.task_execution()
```

3、分析用户意图进行任务拆分

核心点：根据用户自然语言输入，识别出以下几个关键字段，用于驱动后续的工具调用和任务流程：

用户输入	意图分类
获取创建项目接口的结构化测试用例	`生成结构化用例`
执行刚才生成的测试用例	`执行用例`
获取所有接口的测试用例并一键执行	`批量生成并执行结构化用例`
查询“创建项目”接口的定义	`查询接口定义`

1、意图识别提示词

```
# @Author  : 木森
# @weixin: python771

"""意图识别提示词 - 基于自定义工具的智能意图识别系统"""

from langchain_core.prompts import PromptTemplate

intent_recognition_prompt = PromptTemplate(
    input_variables=["user_input", 'tool_info'],
    template="""
    你是一个专业的API自动化测试智能体的意图识别模块。

    ## 系统能力概述
    你拥有以下核心工具，能够完成完整的API测试自动化流程：
    {tool_info}

    ## 意图识别任务
    请分析用户输入，识别用户的真实意图，并根据工具调度链路返回标准化的JSON格式结果。
    每个意图类型都标注了对应的工具调用链路，请严格按照链路顺序进行工具调度。

    ### 支持的意图类型：
    #### 数据查询类 (QUERY)
    - **search_interface_doc**: 向量数据库语义检索，接口文档 (使用 vector_retriever)
    - **query_interface**: 精准查询接口信息 (使用 interface_db_query)
    - **query_base_case**: 查询基础用例 (使用 base_test_case_query)
    - **query_structured_case**: 查询结构化用例 (使用 structured_test_case_query)

    #### 用例生成类 (GENERATE)
```

- **base_case_generation**: 单个接口生成基础用例 (链路: interface_db_query → base_case_generation)
- **one_structured_case_generation**: 基于基础用例生成结构化用例 (链路: base_test_case_query → one_structured_case_generation)
- **generate_complete_case**: 单接口生成基础和结构化用例 (链路 interface_db_query -> complete_case_generation)
- **project_batch_generate**: 项目或模块级 多接口批量用例生成 (链路: interface_db_query → 循环(complete_case_generation))

测试执行类 (EXECUTE)
- **execute_single_case**: 执行单个用例 (链路: structured_test_case_query → case_execution)
- **execute_batch_case**: 批量执行用例 (链路: structured_test_case_query → case_execution)

复合操作类 (COMPOSITE)
- **generate_and_execute**: 生成并执行单接口 (链路: interface_db_query → complete_case_generation → case_execution)
- **project_generate_and_execute**: 项目级或模块级 多接口用例生成并执行 (链路: interface_db_query → 循环(complete_case_generation) → case_execution)

用户输入分析
用户输入: {user_input}

分析要求
1. 仔细分析用户的真实需求和意图
2. 识别关键参数和缺失信息
3. 根据工具调度链路制定合理的执行计划
4. 确保工具调用顺序符合依赖关系
5. 特别注意区分单接口操作和项目级批量操作

返回格式
请返回以下JSON格式的结果（不要有多余解释）：

```json
{{
    "intent_type": "具体的意图类型",
    "task_category": "任务大类(QUERY/GENERATE/EXECUTE/COMPOSITE)",
    "confidence": 0.95,
    "primary_tool": "主要使用的工具名称",
    "tool_chain": ["工具调用链路，按顺序列出"],
    "is_batch_operation": true/false,
    "scope": "single_interface/project_level/module_level",
    "parameters": {{
        "project": "项目名称",
        "project_name": "项目名称",
        "module": "模块名称",
        "interface_name": "接口名称",
        "interface_id": "接口ID",
        "case_id": "用例ID",
        "base_case_id": "基础用例ID",
        "method": "HTTP方法(GET/POST/PUT/DELETE)",
        "tag": "接口标签",
        "status": "用例状态(pending/generated/failed)",
        "review_status": "审核状态(pending/通过/失败/错误)",
        "query": "搜索关键词",
        "k": 5,
```

```
            "search_type": "检索类型(hybrid/bm25/vector)",
            "alpha": 0.6,
            "collection_name": "webapi",
            "test_data": {{}},
            "additional_info": ""
        }},
        "missing_params": ["缺失的关键参数"],
        "execution_plan": [
            {{
                "step": 1,
                "tool_name": "工具名称",
                "action": "具体操作描述",
                "required_params": ["必需参数"],
                "optional_params": ["可选参数"],
                "expected_output": "预期输出描述",
                "depends_on": "依赖的前一步骤输出",
                "is_loop": true/false,
                "loop_condition": "循环条件描述"
            }}
        ],
        "workflow_type": "single_tool/sequential/parallel/conditional/loop",
        "estimated_time": "预估执行时间",
        "risk_level": "low/medium/high"
    }}
    ```

    ## 特殊处理规则
    1. **工具链路验证**: 确保execution_plan中的工具调用顺序符合依赖关系
    2. **参数传递**: 明确指出哪些参数需要从前一步骤的输出中获取
    3. **批量操作识别**:
       - 关键词："所有接口"、"整个项目"、"批量生成"、"项目级"
       - 关键词："所有用例"、"批量执行"
       - 关键词："单个接口"、"某个接口"
    4. **循环处理**: 对于项目级或多个接口操作操作，在execution_plan中标明循环步骤和循环条件
    5. **错误处理**: 对于可能失败的步骤，在execution_plan中标明风险点
    6. **参数缺失**: 如果关键参数缺失，在missing_params中列出，并在execution_plan中标明获取方式
    7. **置信度评估**:
       - 0.9-1.0: 意图明确，参数完整
       - 0.7-0.9: 意图明确，参数部分缺失
       - 0.5-0.7: 意图模糊，需要进一步确认
       - <0.5: 无法识别意图
    """
)
```

## 2、意图识别结果提取

```python
from pydantic import BaseModel, Field
from typing import List, Dict

class IntentRecognitionResult(BaseModel):
    """意图识别结果模型"""
    intent_type: str = Field(..., description="具体的意图类型")
    task_category: str = Field(..., description="任务大类")
```
```

```
    confidence: float = Field(..., ge=0.0, le=1.0, description="置信度")
    primary_tool: str = Field(..., description="主要使用的工具名称")
    tool_chain: List = Field(default_factory=list, description="工具调用链路，按顺序列出")
    is_batch_operation: bool = Field(False, description="是否为批量操作")
    scope: str = Field(..., description="操作范围")
    parameters: Dict = Field(..., description="参数信息")
    missing_params: List = Field(default_factory=list, description="缺失的关键参数")
    execution_plan: List = Field(default_factory=list, description="执行计划")
    workflow_type: str = Field(..., description="工作流类型")
    estimated_time: str = Field(..., description="预估执行时间")
    risk_level: str = Field(..., description="风险等级")
```

3、获取工具信息

```
def get_tools_info(self):
        """获取工具的信息"""
        # 初始化所有工具实例
        tools_info = []
        for tool in self.tools.values():
            tool_info = {
                "name": tool.name,
                "description": tool.description,
                "args_schema": tool.args_schema.model_json_schema() if tool.args_schema else None
            }
            tools_info.append(tool_info)
        return tools_info
```

4、意图识别节点代码

```
from prompts.agent_prompt.intent_recognitio_prompt import intent_recognition_prompt
from agents.models import qwen3_llm, dpr1_0528, deepseek_v3_llm
from tools.case_run.case_run import CaseExecutionTool
from tools.data_query.vec_retriever_tool import VectorRetrieverTool
from tools.data_query.db_query_tools import InterfaceQueryTool, BaseTestCaseQueryTool, StructuredTestCaseQueryTool
from tools.generator_case.compete_case_generation_tool import CompleteCaseGenerationTool
from tools.generator_case.base_case_generation_tool import BaseCaseGeneratorTool
from tools.generator_case.all_case_strutured_tool import BatchStructuredCaseTool
from tools.generator_case.one_case_strutured_tool import OneStructuredCaseGenerationTool
from langchain_core.output_parsers import JsonOutputParser

from pydantic import BaseModel, Field
from typing import List, Dict


class IntentRecognitionResult(BaseModel):
    """意图识别结果模型"""
    intent_type: str = Field(..., description="具体的意图类型")
    task_category: str = Field(..., description="任务大类")
```

```
    confidence: float = Field(..., ge=0.0, le=1.0, description="置信度")
    primary_tool: str = Field(..., description="主要使用的工具名称")
    tool_chain: List = Field(default_factory=list, description="工具调用链路，按顺序列出")
    is_batch_operation: bool = Field(False, description="是否为批量操作")
    scope: str = Field(..., description="操作范围")
    parameters: Dict = Field(..., description="参数信息")
    missing_params: List = Field(default_factory=list, description="缺失的关键参数")
    execution_plan: List = Field(default_factory=list, description="执行计划")
    workflow_type: str = Field(..., description="工作流类型")
    estimated_time: str = Field(..., description="预估执行时间")
    risk_level: str = Field(..., description="风险等级")


class AgentApp:
    """自定义的Agent"""
    parser = JsonOutputParser(pydantic_schema=IntentRecognitionResult)

    def __init__(self, user_input):
        # 用户的初始输入
        self.user_input = user_input
        # 意图识别的公共参数，在意图识别后会被设置
        self.global_parameters = {}
        self.tools = {
            # 数据查询工具
            "vector_retriever": VectorRetrieverTool(),
            "interface_db_query": InterfaceQueryTool(),
            "base_test_case_query": BaseTestCaseQueryTool(),
            "structured_test_case_query": StructuredTestCaseQueryTool(),
            # 用例生成工具
            "complete_case_generation": CompleteCaseGenerationTool(),
            "base_case_generation": BaseCaseGeneratorTool(),
            "batch_structured_case": BatchStructuredCaseTool(),
            "one_structured_case_generation": OneStructuredCaseGenerationTool(),
            # 用例执行工具
            "case_execution": CaseExecutionTool(),
        }
        # 存放执行的任务栈
        self.task_stack = []
        # 绑定工具的大模型
        self.llm_with_tools = deepseek_v3_llm.bind_tools(list(self.tools.values()))

        # 全局的公共参数 parameters
        self.global_parameters = {}

    def get_tools_info(self):
        """获取工具的信息"""
        # 初始化所有工具实例
        tools_info = []
        for tool in self.tools.values():
            tool_info = {
                "name": tool.name,
                "description": tool.description,
                "args_schema": tool.args_schema.model_json_schema() if tool.args_schema else None
            }
```

```
            tools_info.append(tool_info)
        return tools_info

    def intent_recognition(self):
        """意图识别函数"""
        tools_info = self.get_tools_info()
        chain = intent_recognition_prompt | deepseek_v3_llm | self.intent_parser
        # 进行意图识别
        intent_result = chain.invoke({"user_input": self.user_input,
"tool_info": tools_info})
        self.intent_result = intent_result
        print("意图识别结果:", intent_result)
```

4、添加记忆机制

Agent在执行过程中，会与大模型进行多轮对话，所以需要使用记忆机制来记住对话的上下文

1、初始化记忆

```
    def __init__(self, user_input):
        # 用户的初始输入
        self.user_input = user_input
        # 意图识别的公共参数，在意图识别后会被设置
        self.global_parameters = {}

        # 初始化InMemoryStore
        self.store = InMemoryStore()
        self.thread_id = f"agent_session_{hash(user_input) % 10000}"
        self.user_id = "default_user"  # 可以根据需要设置用户ID
```

2、记忆保存

```
    def save_to_memory(self, memory_data: Dict):
        """保存信息到InMemoryStore"""
        try:
            # 定义命名空间
            namespace = (self.user_id, self.thread_id, "memories")
            # 生成唯一键
            memory_id = str(uuid.uuid4())
            # 保存到store
            self.store.put(namespace, memory_id, memory_data)
        except Exception as e:
            print(f"⚠ 保存记忆失败: {e}")
```

3、记忆获取

```
    def get_memory_context(self) -> Dict:
        """从InMemoryStore中获取上下文信息"""
        try:
            # 定义命名空间
            namespace = (self.user_id, self.thread_id, "memories")
```

```
            # 搜索所有相关记忆
            memories = self.store.search(namespace)
            # 分类记忆
            intent_recognition = None
            execution_history = []
            for memory in memories:
                memory_type = memory.value.get("type")
                if memory_type == "intent_recognition":
                    intent_recognition = memory.value.get("content", {})
                elif memory_type == "tool_execution":
                    execution_history.append(memory.value.get("content", {}))
            # 只保留最近的执行历史
            execution_history = execution_history[-10:]
            return {
                "intent_recognition": intent_recognition,
                "execution_history": execution_history,
            }
        except Exception as e:
            print(f"⚠ 获取记忆上下文失败: {e}")
            return {
                "intent_recognition": None,
                "execution_history": [],
            }
```

5、function calling调用

1、function calling提示词

```
    def get_func_calling_prompt(self, task):
        """
        构建func_calling的提示词的组成:
            执行的任务
            上下文(从记忆中获取)
                初始需求:
                意图识别的结果:
                历史工具执行的结果
        :param tools_result:
        :return:
        """

        prompt = f"""
        用户需求: {self.user_input}
        当前任务: {task.get('action')}
        调用的工具: {task.get('tool_name')}
        可用的参数数据列表:
        {self.intent_result.get('parameters')}
        历史执行的数据
        {self.get_memory()}
        """
        return prompt
```

2、工具调用

```python
def task_execution(self):
        """任务执行的函数"""
        while self.task_queue:
            # 取出要执行的任务
            task = self.task_queue.pop()
            print(
                f"====正在执行步骤{task.get('step')},具体操作描述:
{task.get('action')},执行的工具: {task.get('tool_name')}====")
            func_prompt = self.get_func_calling_prompt(task)
            # 使用大模型function calling模型去根据上下文构建工具调用
            tools_result = self.llm_with_tools.invoke(func_prompt)
            # 获取要执行的工具
            if hasattr(tools_result, "tool_calls") and tools_result.tool_calls:
                # 获取要执行的工具
                for tool in tools_result.tool_calls:
                    # 获取工具名称
                    tool_name = tool.get('name', '')
                    # 获取工具参数
                    tool_args = tool.get('args', '')
                    # 实际的工具执行
                    result = self.tools[tool_name].invoke(tool_args)
                    print(f"步骤{task.get('action')}执行完毕")
                    print(f"工具{tool_name}执行的结果: ", result)
                    # 将工具执行的结果加入到记忆中
                    self.save_to_memory({
                        "role": "assistant",
                        "content": result,
                        "type": "tool_execution",
                        "tool_name": tool_name,
                        "tool_args": tool_args
                    })
```